Modelling and Control of the Flexibac Robotic Sorting System: A Multi-Approach Study Using FlexSim

Abir Ben Bouzaiene, Olivier Boutin, and Pascal André

Abstract This study explores three distinct modelling approaches to address the scheduling and control challenges of the 2025 *IMIC* robotic system, all of them based on the same production management algorithm. The *IMIC* is a case-study contest that is taking place in the context of the *SOHOMA* workshop series. All of these solutions are meant to be run eventually within the *FlexSim* piece of software, as stated in the contest rules. We first implemented a data-driven simulation based on the input files provided by the organisers of the contest, an approach we selected for our official contest entry due to its scenario-testing flexibility. Subsequently, we developed an embedded control logic within *FlexSim* and finally a real-time external optimization approach connecting a Python script with *FlexSim* via socket communication. Our goal is to compare these methods and to highlight their strengths and limitations. The analysis demonstrates that, while each approach offers unique advantages tailored to specific operational needs, they also present different trade-offs regarding ease of integration, computational complexity, and adaptability to dynamic production environments. By examining these methods side by side, this work provides valuable insights into how each modelling strategy can be leveraged to control a complex system. For our official contest submission, we adopted the output files of the data-driven simulation.

Abir Ben Bouzaiene
Polytech intl, LAC1 Tunis, Tunisia, e-mail: a.benbouzaiene22196@pi.tn

Olivier Boutin
3iL Ingénieurs – campus de Nantes, F-44800 Saint-Herblain, France and LS2N, UMR 6004, F-44000 Nantes, France, e-mail: olivier.boutin@ls2n.fr

Pascal André
Nantes Université, École Centrale Nantes, CNRS, LS2N, UMR 6004, F-44000 Nantes, France, e-mail: pascal.andre@ls2n.fr

1 Project Context, Objectives and Methodological Framework

This section introduces the context of the research project, the main objectives, and the methodology adopted throughout this work. The project is based on a case study that was designed for the 2025 *IMIC*[1] happening in the scope of the next *SOHOMA*[2] workshop to come[3], which features a simplified system in which a robotic arm is responsible for moving boxes to given carts, in order to comply with production management objectives [Klement et al., 2024].

While the original problem was intended to be competition challenge, our objective in this publication extends beyond simply finding a solution to the provided scenario. In this study, the case is used as a foundation to explore and compare different modelling and control strategies within *FlexSim.*

1.1 The Flexibac *System*

The foundation of this study is the *Flexibac* system, developed through a research collaboration between *La Poste* (the French postal service) and *Nantes Université*. The *Flexibac* system has been as an inspiration for the 2025 edition of the *IMIC*, which provided participants a simplified representation of a real-world industrial problem centred around the automation of mail sorting operations.

At the heart of the system is a 6-axis robotic arm, designed to automate the physical handling of heavy mail containers. This robotic solution aims at reducing operator strain by managing the transfer of boxes into designated carts.

The readers are invited to consult the official *IMIC* Website to get more insight on the problem core components, rules and parameters as well as the plant logical layout [Klement et al., 2025].

1.2 Study Objectives

In approaching the *Flexibac* system challenge, our work is guided by both the practical requirements of the contest and a broader interest in methodological exploration. The two main goals of this study are as follows.

---

[1] Intelligent Manufacturing International Contest

[2] Service-Oriented, Holonic and Multi-Agent Manufacturing Systems for Industry of the Future

[3] `sohoma2025.sciencesconf.org`

1. **Provide a practical solution for the *IMIC* contest** to develop a feasible and effective solution addressing the scheduling and control challenges of the *Flexibac* system, as presented in the *IMIC* contest case study.
2. **Explore and compare different modelling approaches** to experiment with various simulation and optimization methods in order to assess their respective strengths, limitations, and suitability for different operational scenarios.

Together, these aims enable us not only to meet the contest requirements but also to gain broader insights into modelling techniques applicable to complex automated systems.

1.3 Methodology

To address the scheduling and control challenges of the *Flexibac* system and meet the study objectives, we developed and implemented three distinct modelling approaches.

1. **Data-driven simulation**: in this approach, the input files that depend on the production management logic are generated by a Python script based on system parameters and scenario data. These files are then manually imported into the *FlexSim* model to drive the simulation and control the system's operations according to predefined schedules and constraints.
2. **Embedded control logic**: the system's operational rules and optimization algorithms are directly programmed within the *FlexSim* environment. This integration allows the simulation to run autonomously, with all control decisions made internally without the need for external input or communication.
3. **Socket-based communication**: this method establishes a real-time communication link between *FlexSim* and an external Python-based optimizer through socket connections. The external optimizer dynamically sends control commands to *FlexSim* during the simulation run, allowing for interactive scheduling and control adjustments based on current system states.

These three approaches provide a comprehensive framework for exploring different modelling and control strategies within the *Flexibac* system. The following section will focus on the practical implementation of these solutions within the *FlexSim* environment, providing detailed descriptions and results for each method.

2 Implementation of Modelling Approaches

This section introduces the core production management logic, followed by a detailed explanation of its implementation across all approaches.

2.1 Core Production Management Logic

The control logic behind the *Flexibac* system revolves around the efficient assignment and management of carts based on box destination codes, within the constraints of the system's capacity and timing parameters. At the core of the solution is the parameter `Nc`, representing the maximum number of carts that can be positioned around the robot at any given time. Each cart is assigned a single destination and can hold up to `Qmax` boxes.

Our solution to optimize the sorting process involves identifying the top (most frequent) destinations among the boxes entering the system. These top destinations are selected based on the current distribution of incoming box destinations in **Buffer A** and the number of available carts (`Nc`) around the robot.

Once the top destinations are determined, each one is assigned to a cart. Boxes with destination codes matching these top destinations are moved to **Buffer B**, where they await pickup by the robot in a First-In, First-Out (FIFO) fashion.

In contrast, boxes with non-prioritized destinations (those not among the top selection) are redirected to the **Manual storage area** to be handled separately through manual processing. As for the robotic arm, it picks boxes from **Buffer B** one by one and places each box into the appropriate cart based on its destination code.

Cart changeover occurs under the two following conditions.

1. **Full cart**: when a cart reaches its maximum capacity (`Qmax`), it is removed and replaced with a new cart with the same destination.
2. **End of scenario**: if the simulation reaches the defined scenario time limit (`EndDate`), any remaining partially filled cart is also considered for unloading.

To better understand the optimization logic behind the *Flexibac* system, Algorithm 1 provides a more formal way to explain it, written in pseudocode.

Although this solution may seem straightforward at first glance, it is logical, robust and efficient. Its simplicity allows for easy implementation and adaptability, which is especially important given our objective to apply it across multiple modelling and control approaches.

```
input  : Input.dat file
output: BoxTransfer.dat and CartTransfer.dat files
identify the top destinations from Input.dat;
assign these destination to available carts;
while global time has not reached EndDate do
    for each incoming box from Input.dat taken in chronological order do
        if destination corresponds to one of the carts then
            send to Buffer B;
        else
            send to manual storage;
        end
    end
    while Buffer B is not empty do
        take the next box (FIFO order);
        pop that box out of Buffer B;
        put the box into the right cart;
        if cart is full then
            unload the cart;
            replace it with and empty cart, with the same destination;
        end
    end
    unload all non-empty carts;
end
```

Algorithm 1: Pseudocode of the *Flexibac* optimization algorithm

2.2 Data-Driven Simulation Model

This approach relies on simulating the *Flexibac* system's operations externally through structured data files. The core idea is to base the simulation on real or realistically modelled data, capturing the system's dynamics without embedding control logic directly into the simulation software.
A Python script is developed to serve as the decision-making engine, analysing input data and generating the required instructions for the simulation.
The process begins with the two following main input files:

- **Input.dat**, which contains raw data about box arrivals, including arrival times, destination codes and identifiers;
- **Parameters.dat**, which defines the simulation parameters such as the number of carts available (`Nc`), the cart capacity (`Qmax`) and the simulation end date (`EndDate`).

Using this information, the Python code processes the data, identifies the top destinations and applies the cart assignment logic while respecting the system constraints (FIFO, homogeneity of carts, scenario timing).

As a result, the following two output files are generated:

- **BoxTransfer.dat**, which specifies when and where each box should be transferred, including its assignment to the robotized storage area or the manual area;

- **CartTransfer.dat**, which provides instructions for cart movements, including when a cart should be replaced based on either reaching capacity or the scenario end.

All four files (`Input.dat`, `Parameters.dat`, `BoxTransfer.dat` and `CartTransfer.dat`) are then manually imported into the *FlexSim* benchmark model designed to compare all the solutions provided by the contest competitors. Inside *FlexSim*, the model reads these files to control the simulation in alignment with the generated plan. The corresponding Python code implementing Algorithm 1 is provided in Appendix B of this document.

By the way, let us note that this method separates the control logic from the simulation, through emulation [Boutin et al., 2007], making it easy to test different scheduling scenarios by simply changing the input files without having to modify the main *FlexSim* model that is then mainly dedicated to the modelling of the physical part of the system.

2.3 Embedded Control Logic in FlexSim

In this approach, the control logic is fully implemented within the *FlexSim* environment using built-in tools such as "Process Flow" and task executors [FlexSim, 2021]. It eliminates the need for external scripts or communication, making it a straightforward and self-contained solution.

To run a scenario, few manual steps are required:

1. box arrival times are entered using the Schedule Source, which defines when boxes enter the system;
2. the model calculates the top destinations based on the distribution of destinations in Buffer A and the number of available carts around the robot;
3. destinations are then assigned to carts using embedded scripts in the Process Flow elements. These scripts can be easily adjusted whenever the scenario changes;
4. boxes are collected into carts using a Batch element. When the number of boxes reaches `Qmax` or the current time reaches `EndDate`, the Batch element is triggered.

This approach is simple to set up, quick to test, and does not require any external development effort. All logic is handled inside *FlexSim*, making it convenient for building, adjusting and running different scenarios within the same simulation environment.

2.4 Real-Time Optimization via Socket Communication

This approach integrates an external Python-based optimizer with the *FlexSim* simulation model using real-time **socket communication** between so-called server and client. The goal is to enable dynamic decision-making and adaptive control by allowing *FlexSim* to exchange data with the external optimizer during the simulation run.

2.4.1 Architecture and Communication Setup

Figure 1, inspired by [Leon et al., 2022], illustrates the communication framework between Python (server) and *FlexSim* (client).

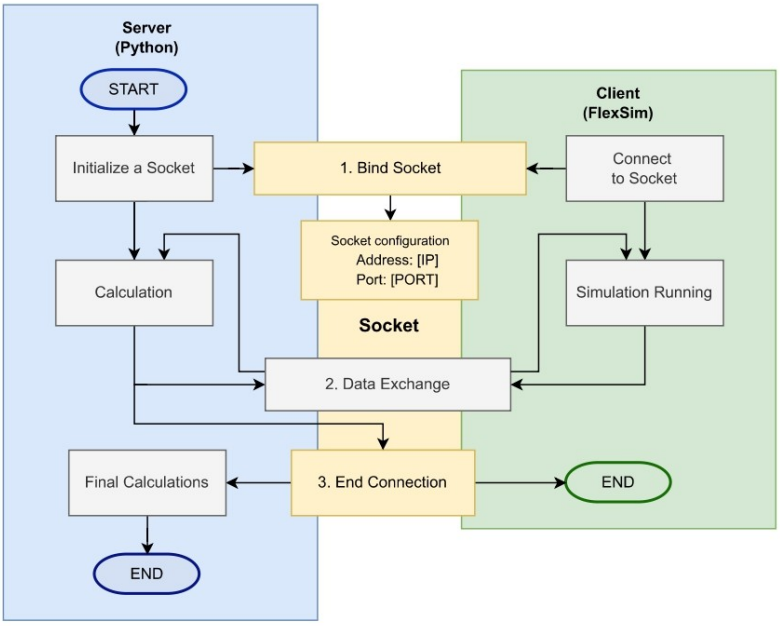

Fig. 1 Client-Server communication

The system operates following a **client-server** architecture:

- **a socket is initialized** between the client and server, specifying IP address and port number for communication;
- ***FlexSim* acts as the client**, initiating communication and sending system state data;

- **a Python script acts as the server**, running an optimization algorithm that receives the data, computes decisions and sends the result back to *FlexSim*;
- through the connection lifecycle the socket remains active for multiple **request-response cycles** until simulation completion.

The communication occurs through **TCP sockets**[4], enabling fast and stable data transfer during runtime and works fine between running instances of *FlexSim* and Python code [Leon et al., 2022].

This setup allows the simulation model to react dynamically to system changes, such as variations in arrival patterns or changes in cart availability.

2.4.2 Workflow Overview

This approach follows a structured workflow that ensures seamless communication as describe below.

1. **System state retrieval**: during the simulation, *FlexSim* collects real-time information such as the number and destination of all boxes in Buffer A, the availability of carts and the current global time in the simulation.
2. **Sending data to Python**: this information is packed into a message following a specific syntax and sent via a socket to the Python script.
3. **External optimization**: the Python server receives the data and executes the optimization algorithm.
4. **Returning results**: once decisions are made, the results are sent back to *FlexSim*, following the expected message syntax.
5. ***FlexSim* execution**: *FlexSim* applies the optimizer's decisions in real-time, continuing the simulation accordingly.

This method was the most complex to implement, requiring close coordination between the simulation and external optimizer. It enables smart, adaptive control and lets us evaluate performance in realistic, changing conditions just like in a modern industrial system. Similar integrated approaches have proven effective in related industrial optimization problems, as shown in [Leon et al., 2022].

[4] Transmission Control Protocol sockets are endpoints for bidirectional communication between two devices over a network.

3 Comparative Analysis

3.1 Evaluation Criteria

To assess the effectiveness of each modelling approach, we defined a set of evaluation criteria. The criteria include:

- **simulation outcome**, which assesses whether the model successfully provided the expected system behaviour, primarily focusing on the number of robotized and manually handled boxes and the number of carts changes. Since the same logic was applied in each method, this metric serves as a validation of correctness and not as a point of differentiation;
- **implementation complexity**, which measures the practical effort and technical difficulty involved in developing and maintaining the solution. This includes setup requirements, coding needs and how much manual intervention is required;
- **model flexibility**, which reflects how easily the model can be adapted to new scenarios, modified parameters or extended functionalities. This includes how changes to the control logic or input data are handled within each framework;
- **scalability**, which this assesses how well the model handles increasing system complexity such as more destinations or higher box arrival rates.

These criteria provide a balanced view, capturing both operational outcomes and modelling experience.

3.2 Comparative Results

The table below summarizes the performance of the three approaches based on the defined criteria.

Table 1 Comparison of the three approaches

Criteria	**Data-Driven**	**Embedded**	**Socket-Based**
System Performance	Same	Same	Same
Implementation Effort	Medium	Low	High
Model Flexibility	High	Low	High
Scalability	Moderate	Low	High

All the approaches yielded consistent **simulation outcomes**, meaning that the key performance indicators (KPIs) such as the number of cart

changes and the distribution between robotized and manually handled boxes remained identical across the different implementations.

This was expected, as they all implemented the same core control logic. The similarity in system behaviour across the models confirms that the logic was correctly applied in each case, allowing us to focus our comparison on practical aspects beyond performance results.

When evaluating **implementation complexity**, the embedded control logic was the easiest to implement. It required minimal setup and could be created entirely within the *FlexSim* environment using basic scripting. The data-driven method involved more effort upfront to generate and manage input/output files but remained manageable. In contrast, the real-time socket communication approach was the most complex to implement. It demanded precise synchronization between the Python optimizer and *FlexSim* which involved developing scripts on both sides: in *FlexSim*, to send and decode socket messages, and in Python, to receive, interpret, and respond to those messages appropriately. This also required additional testing to ensure stability.

In terms of **model flexibility**, the socket-based real-time approach proved to be the most adaptable. It enabled dynamic decision-making during simulation and eliminated the need to manually enter input files into the *FlexSim* model, unlike the data-driven method. The data-driven simulation approach was also flexible, as decoupling the control logic from the *FlexSim* environment and placing it in a Python script allowed for rapid testing of different scenarios by modifying input files. The embedded control logic in *FlexSim*, while straightforward, was the least flexible; scenario changes required manual updates to the process flow and scripts.

Regarding **scalability**, the socket-based solution stood out. Its externalized control logic and dynamic feedback loop allowed it to handle more complex scenarios without modifying the simulation structure. The data-driven model also scaled well, as changes could be managed in the Python layer. However, the embedded approach showed limitations as the system grew more complex, making it harder to manage changes and increasing the risk of script errors.

3.3 Selected Approach and Results

The selected solution for the *IMIC* contest is the one based on the **data-driven simulation** approach. This choice was motivated by the contest's emphasis on generating and verifying results through distinct data files, which facilitates a transparent and reproducible evaluation process. Additionally, this method offers practical advantages in terms of ease of modification and adaptability, allowing for straightforward updates and testing of alternative scenarios.

The following figures illustrate the performance comparison between our implemented solution and the organizers reference .

Figure 2 presents the KPIs obtained from the implemented approaches (results are identical across all three approaches) while Figure 3 presents the KPIs from the reference solution provided by the organizers. Both figures are captured from the dashboard interface dedicated to the contest KPIs that was provided in the *FlexSim* benchmark model.

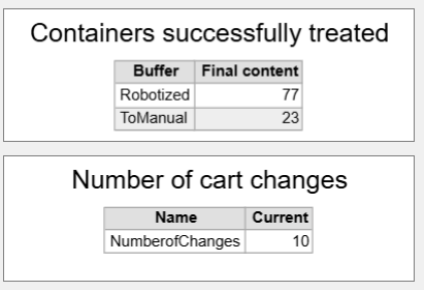

Fig. 2 Our implemented solution KPIs

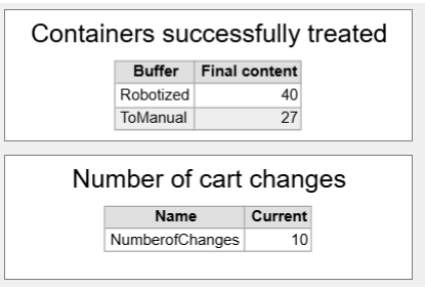

Fig. 3 Reference solution KPIs

The comparison highlights that our proposed solution outperforms the organizers reference model in several key areas. Firstly, the number of treated boxes is higher, indicating improved system throughput. Additionally, our model achieves a greater proportion of robotized boxes, reducing reliance on manual handling and increasing automation efficiency. Notably, although our solution achieves higher throughput, the number of cart changes remains the same as the reference model, highlighting the efficiency of our approach in handling increased workload without additional cart reassignments.

3.4 Lessons Learned and Best Practices

Through this study, it became clear that choosing the right modelling approach depends heavily on the specific needs and goals of the project. The embedded control logic is simple to implement and works well when quick setup and low complexity are priorities. However, it may not offer the flexibility needed for frequent changes or complex optimization.

The data-driven simulation method provides more flexibility, allowing easy testing of different scenarios by simply adjusting input files without changing the core model. This makes it useful for exploring "what-if" situations and understanding system behaviour under varying conditions.

The real-time socket communication approach, while the most complicated to implement, is the best fit for systems requiring smart, adaptive control that

reacts dynamically to real-time information. In fact, it paves the way for a control in a dioid framework [Brunsch et al., 2013, Hardouin et al., 2013], for which we need a scalable and robust approach so as to be implemented on real world systems. For instance, the specifications dealing with capacities (buffer size constraints, number of boxes in a cart and, maybe, even the maximum number of carts in use at a time) and synchronisations between system sub-parts should be manageable within the context of controlled-invariance [Martinez et al., 2022]. The latter has actually been implemented in Python within the PyMinMaxGD library [Boutin et al., 2024] and is consequently compatible with the socket communication protocol designed for this study. This approach reflects modern industrial requirements but demands more coordination and technical effort.

4 Summary

We first started working on the *Flexibac* system as part of a final year internship research project, which centres on exploring and testing different tools and approaches (beyond just *FlexSim*, like Petri nets modelling and simulation tools). This IMIC report reflects a focused part of that broader project, where we concentrated specifically on applying and analysing solutions within the *FlexSim* environment.

In this study, we examined the *Flexibac* robotic sorting system through three main approaches: data-driven simulation, embedded control logic, and real-time optimization via socket communication. Each approach brings its own balance of flexibility, complexity, and adaptability. The data-driven simulation, which we selected for the IMIC contest application, offers ease of scenario testing and modularity, while the embedded control logic is straightforward and requires minimal effort to implement. The real-time socket communication method, though the most complex, enables dynamic and intelligent control representative of modern industrial systems.

The Python code used to produce the contest submission files has been included in a dedicated appendix for the sake of reproducibility of our results. The *FlexSim* files for the two other studied simulation architectures can be provided on demand, by contacting the authors by e-mail.

By comparing these approaches across simulation results, flexibility, implementation effort, and scalability, we gained valuable insights into their practical strengths and limitations. Ultimately, the choice of approach depends on the specific goals and needs of the project, highlighting the importance of selecting the right tools to ensure effective and sustainable system control.

5 Acknowledgements

A References

[Boutin et al., 2007] Boutin, O., Cottenceau, B., and L'Anton, A. (2007). Online Control of a (max,+) Linear Emulated Production System. In [Yang et al., 2007], page 321.

[Boutin et al., 2024] Boutin, O., Martinez, C., and Rakoto, N. (2024). On Solving Controlled-Invariance Problems in Dioids Using the PyMinMaxGD Python Scripts Library. In *Proceedings of the 21st International Conference on Informatics in Control, Automation and Robotics (ICINCO 2024)*, volume 1, pages 555 – 566, Porto, Portugal. INSTICC.

[Brunsch et al., 2013] Brunsch, T., Raisch, J., Hardouin, L., and Boutin, O. (2013). Discrete-Event Systems in a Dioid Framework: Modeling and Analysis. In [Seatzu et al., 2013], chapter 21, pages 431 – 450. Available at `dx.doi.org/10.1007/978-1-4471-4276-8_21`.

[FlexSim, 2021] FlexSim (2021). Introduction to model logic. `docs.flexsim.com/en/21.0/ModelLogic/IntroModelLogic/IntroModelLogic.html`. Retrieved on 30 July 2025.

[Hardouin et al., 2013] Hardouin, L., Boutin, O., Cottenceau, B., Brunsch, T., and Raisch, J. (2013). Discrete-Event Systems in a Dioid Framework: Control Theory. In [Seatzu et al., 2013], chapter 22, pages 451 – 469. Available at `dx.doi.org/10.1007/978-1-4471-4276-8_22`.

[Klement et al., 2024] Klement, N., Haddou Benderbal, H., Derigent, W., and Cardin, O. (2024). IMIC: Intelligent Manufacturing International Contest - towards the gradual design of a new benchmark. In *14th International Workshop on Service-Oriented, Holonic and Multi-Agent Manufacturing Systems for Industry of the Future, SOHOMA 2024*, Augsburg, Germany.

[Klement et al., 2025] Klement, N., Haddou Benderbal, H., Derigent, W., and Cardin, O. (2025). Benchmark 9. Intelligent Manufacturing International Contest: IMIC. `github.com/GIS-S-mart/Benchmark-9-IMIC/`. Retrieved on 30 July 2025.

[Leon et al., 2022] Leon, J. F., Marone, P., Peyman, M., Li, Y., Calvet, L., Dehghanimohammadabadi, M., and Juan, A. A. (2022). A Tutorial on Combining Flexsim with Python for Developing Discrete-Event simheuristics. In *2022 Winter Simulation Conference (WSC)*, pages 1386 – 1400. IEEE.

[Martinez et al., 2022] Martinez, C., Kara, R., Abdesselam, A. N., and Loiseau, J. J. (2022). Systems synchronisation in Max-Plus algebra: a controlled invariance perspective *In memoriam Édouard Wagneur.* In *1st IFAC Workshop on Control of Complex Systems*, Bologna, Italy.

[Seatzu et al., 2013] Seatzu, C., Silva, M., and van Schuppen, J. H., editors (2013). *Control of Discrete-Event Systems: Automata and Petri Net Perspectives*, volume 433 of *Lecture Notes in Control and Information Sciences*. Springer, London.

[Yang et al., 2007] Yang, S., Chen, G., Thomas, A., Artiba, A., and Xu, Z., editors (2007). *Proceedings of the 2nd International Conference on Industrial Engineering and Systems Management, IESM'07*, Beijing, China. Tsinghua University Press.

B Python code for Data-Driven Simulation Model

```
"""
Copyright 2025 Abir Ben Bouzaiene

This program is free software: you can redistribute it and/or modify it
under the terms of the GNU General Public License as published by
the Free Software Foundation, either version 3 of the License,
or (at your option) any later version.

This program is distributed in the hope that it will be useful,
but WITHOUT ANY WARRANTY; without even the implied warranty of
MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See
the GNU General Public License for more details.

You should have received a copy of the GNU General Public License
along with this program. If not, see <https://www.gnu.org/licenses/>.
"""

import pandas as pd
import random

# File paths
input_file_path = "C:/Users/benbo/Downloads/FinalResult2/Input.dat"
parameters_file_path = "C:/Users/benbo/Downloads/FinalResult2/Parameters.dat"
box_transfer_path = "C:/Users/benbo/Downloads/FinalResult2/BoxTransfer.dat"
cart_transfer_path = "C:/Users/benbo/Downloads/FinalResult2/CartTransfer.dat"
calendar_output_path = "C:/Users/benbo/Downloads/FinalResult2/Calendar.dat"

# --- Read parameters ---
params = pd.read_csv(
    parameters_file_path,
    sep=";",
    header=0,
    usecols=[0, 1],
    names=["Parameter", "Value"],
)
params_dict = dict(zip(params["Parameter"], params["Value"]))

Bmax = int(params_dict["Bmax"])
Tp = int(params_dict["Tp"])
Qmax = int(params_dict["Qmax"])
Tc = int(params_dict["Tc"])
EndDate = int(params_dict["EndDate"])
Nc = int(params_dict["Nc"])

# --- Read input data ---
df = pd.read_csv(
    input_file_path,
    sep=";",
    header=None,
    usecols=[0, 1, 2],
    names=["ID", "Destination", "Time_instant"],
```

```
)

# Top Nc destinations
top_destinations = df["Destination"].value_counts().head(Nc).index.tolist()

# Assign Type based on destination
df["Type"] = df["Destination"].apply(
    lambda d: "Robot" if d in top_destinations else "Manual"
)

# Save original input (Buffer A times)
original_df = df[["ID", "Destination", "Time_instant"]].copy()
original_df.rename(columns={"Time_instant": "Buffer_A_Time"}, inplace=True)

# --- Robot boxing time with waiting for cart availability ---

# Initialize tracking structures
cart_load = {dest: 0 for dest in top_destinations}
cart_available_time = {dest: 0 for dest in top_destinations}
robot_available_time = 0  # Robot handles one box at a time

# Add Buffer_A_Time to df (from original)
buffer_a_times = original_df.set_index("ID")["Buffer_A_Time"].to_dict()
df["Buffer_A_Time"] = df["ID"].map(buffer_a_times)

# Sort by Buffer_A_Time ascending for sequential processing
df = df.sort_values(by="Buffer_A_Time").reset_index(drop=True)

# Prepare Buffer_B_Time column for robot boxing start time
df["Buffer_B_Time"] = None

for idx, row in df.iterrows():
    dest = row["Destination"]
    buffer_a = row["Buffer_A_Time"]

    if dest in top_destinations:
        # Robot can start boxing only after box arrives,
        # cart is available, and robot is free from previous boxing
        start_boxing = max(buffer_a, cart_available_time[dest], robot_available_time)

        df.at[idx, "Buffer_B_Time"] = start_boxing

        # Update robot availability (boxing duration Tp)
        robot_available_time = start_boxing + Tp

        # Increase cart load for this destination
        cart_load[dest] += 1

        # If cart is full, schedule transfer and block cart
        if cart_load[dest] == Qmax:
            transfer_time = start_boxing + 20  # offset after boxing for transfer
            cart_available_time[dest] = transfer_time + Tc  # cart blocked during Tc

            # Reset cart load after transfer
```

```
            cart_load[dest] = 0
    else:
        # For Manual destinations, boxing time = arrival time
        df.at[idx, "Buffer_B_Time"] = buffer_a

# Convert boxing times to int (optional)
df["Buffer_B_Time"] = df["Buffer_B_Time"].astype(int)

# --- BoxTransfer output ---
df_box = df[["ID", "Buffer_B_Time", "Type"]].copy()
df_box_sorted = df_box.sort_values(by="Buffer_B_Time")  # Sort by boxing start time

with open(box_transfer_path, "w") as f:
    for i, row in enumerate(df_box_sorted.itertuples(index=False, name=None)):
        line = ";".join(map(str, row)) + ";"
        f.write(line + ("\n" if i < len(df_box_sorted) - 1 else ""))

# Assign carts to top destinations
carts = [f"C{i+1}" for i in range(Nc)]
dest_cart_map = {dest: carts[i % Nc] for i, dest in enumerate(top_destinations)}

# --- Box calendar (Buffer A → Buffer B → Cart) ---
df_top = df[df["Destination"].isin(top_destinations)].copy()

df_top["Cart_Time"] = (
    df_top["Buffer_B_Time"] + Tp
)  # Cart loading starts after boxing finishes

calendar_df = df_top[
    ["ID", "Destination", "Buffer_A_Time", "Buffer_B_Time", "Cart_Time"]
]
calendar_df.sort_values(by="Buffer_A_Time", inplace=True)
calendar_df.to_csv(calendar_output_path, index=False, sep=";")

# --- CartTransfer logic with Qmax constraint and Tc blocking ---

cart_rows = []

for dest in top_destinations:
    cart_id = dest_cart_map[dest]
    cart_rows.append([cart_id, 0, dest])  # Start of period record

    dest_boxes = calendar_df[calendar_df["Destination"] == dest].sort_values(
        by="Cart_Time"
    )
    count = 0
    cart_available = 0  # When cart is next available for loading

    for _, row in dest_boxes.iterrows():
        box_time = max(row["Cart_Time"], cart_available)
        count += 1

        if count == Qmax:
            transfer_time = box_time + 20
```

```
            cart_rows.append([cart_id, transfer_time, dest])
            cart_available = transfer_time + Tc
            count = 0

    # Final transfer if any boxes remain in cart at end
    if count > 0:
        cart_rows.append([cart_id, EndDate - 1, dest])

# Write CartTransfer.dat sorted by time
df_cart = pd.DataFrame(cart_rows, columns=["CartID", "Time_instant", "Destination"])
df_cart_sorted = df_cart.sort_values(by="Time_instant")

with open(cart_transfer_path, "w") as f:
    for i, row in enumerate(df_cart_sorted.itertuples(index=False, name=None)):
        line = ";".join(map(str, row)) + ";"
        f.write(line + ("\n" if i < len(df_cart_sorted) - 1 else ""))

# Output summary
box_counts = df["Destination"].value_counts().reset_index()
box_counts.columns = ["Destination", "Box_Count"]
box_counts = box_counts.sort_values(by="Destination")

print("Box counts per destination:\n", box_counts)
print("Top destinations assigned to carts:\n", dest_cart_map)
print("Calendar saved to:", calendar_output_path)
print("BoxTransfer saved to:", box_transfer_path)
print("CartTransfer saved to:", cart_transfer_path)
```